

Основы современного JavaScript для React

Отработка фундаментальных практик ES6+ в изолированной среде: иммутабельный стейт, деструктуризация, методы трансформации списков и безопасные операторы.

`> zsh - node runtime lab - 80x24`

```
# Инициализируем тестовый стенд для отработки ES6+
```

```
→ node -e
```

```
"console.log('✅ Node.js V8 Engine готов к выполнению  
  React-паттернов!')"
```

```
✅ Node.js V8 Engine готов к выполнению React-паттернов!
```

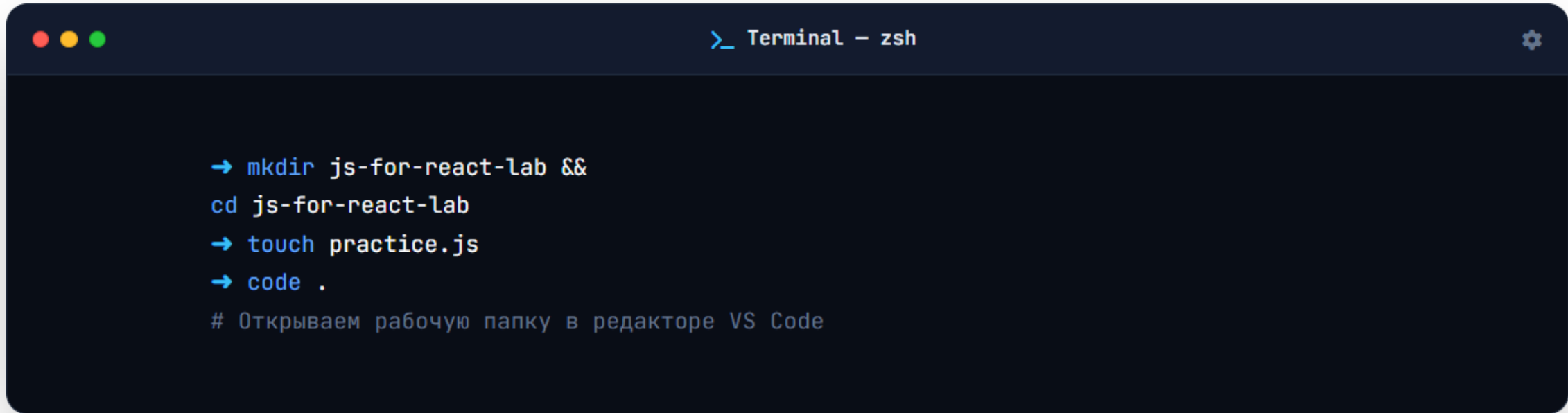
01

Подготовка тестовой среды (Node REPL или JS-файл)

Создание изолированного файла для отработки скриптов перед интеграцией в React

Отрабатывать синтаксис чистого JavaScript удобнее всего в изолированном файле, исполняемом через Node.js.

+ 1.1. Создание рабочей папки и запуск скрипта



```
>_ Terminal - zsh

→ mkdir js-for-react-lab &&
cd js-for-react-lab
→ touch practice.js
→ code .
# Открываем рабочую папку в редакторе VS Code
```

Теперь любой написанный код в файле `practice.js` вы можете мгновенно запустить и проверить командой `node practice.js`.

Стрелочные функции и неявный возврат объектов

Отработка компактного синтаксиса обработчиков событий и генераторов данных

В React стрелочные функции используются для написания функциональных компонентов, обработчиков кликов и генераторов объектов стейта.

</> 2.1. Создание генератора объектов (practice.js)

JS practice.js — Visual Studio Code

```
// 1. Стрелочная функция с неявным возвратом форматированной
// строки:
const
formatCurrency = (amount, currency =
"RUB") =>
`${amount.toLocaleString("ru-RU")} ${currency}`;
```

```
// 2. Неявный возврат объекта (круглые скобки ({ ... })  
    обязательны!):  
  
const  
createProduct = (id, title, price) =>  
  ({ id, title, price, formattedPrice:  
    formatCurrency(price), inStock:  
    true }); console.log(formatCurrency(150000)); // "150 000 RUB" console.log(createProduct(101, "MacBook Air M3",  
    125000));
```

i Почему скобки `{ ... }` обязательны?

Фигурные скобки `{ }` после стрелки `=>` движок JS воспринимает как блок тела функции. Чтобы сообщить компилятору, что мы возвращаем объектный литерал, литерал оборачивается в круглые скобки.

Деструктуризация — основной способ чтения `props` и кортежей хуков в React.

📁 3.1. Эмуляция чтения Props и кортежа useState

```
propsDestructuring.js

// Входной объект (аналог props компонента)
const incomingProps = { id:
  "btn-42", label:
  "Оформить заказ", variant:
  "success", config: { timeout:
    3000 } };

// Извлекаем поля: variant с дефолтом, timeout из вложенного
// объекта:
function
renderButtonProps({ label, variant =
  "primary", config: { timeout } }) {
```

```
console.log(`Кнопка: "${label}" | Стилль: ${variant} | Таймаут:
  ${timeout}мс`); }
```

```
renderButtonProps(incomingProps);
```

```
// Деструктуризация массива (эмуляция хука useState):
```

```
function
```

```
mockUseState(initialValue) {
```

```
  let val = initialValue;
```

```
  const setter = (newVal) => { val =
    newVal; }; return [val, setter]; }
```

```
const [currentCount, setCount] =
```

```
mockUseState(10); console.log("Начальный счетчик:", currentCount); // 10
```

Spread и Rest: Иммуutableное обновление объектов

Копирование данных без мутации исходного состояния и сбор остаточных пропсов

В React запрещено мутировать стейт напрямую (`state.user.age = 26`). Всегда создается новый объект через Spread.

4.1. Проверка иммутабельности и сбор Rest-параметров

```
const initialUser = { id:  
  1, name:  
  "Алексей", role:  
  "Junior Developer"  
};
```

```
// 1. Создаем нового пользователя с обновленной должностью через  
  Spread:
```

```
const promotedUser = {
  ...initialUser, role:
  "Middle React Developer", updatedAt:
  Date.now() };

// 2. Проверяем ссылочную идентичность (как это делает React
// Object.is):
console.log("Ссылки равны?", initialUser === promotedUser);
// false (React увидит обновление!)
console.log("Исходный объект остался цел:", initialUser.role);
// "Junior Developer"

// 3. Сбор остаточных пропсов (Rest):
const { id, role, ...displayData } =
promotedUser; console.log("Остаточные свойства (Rest):", displayData);
// { name: 'Алексей', updatedAt: ... }
```

Каждое действие в React-приложении (добавить в корзину, удалить задачу, переключить чекбокс) выражается через методы массивов.

☰ 5.1. Полный CRUD корзины товаров

```
const initialCart = [ { id:
1, name:
"Клавиатура", price:
4000, quantity:
1 }, { id: 2,
name: "Мышь", price:
2500, quantity:
2 }, { id: 3,
name: "Коврик", price:
1000, quantity:
```

```
1 } ];
```

```
// А. ДОБАВЛЕНИЕ (Add item) → Spread в новый массив:
```

```
const newItem = { id:
```

```
4, name:
```

```
"Наушники", price:
```

```
6000, quantity:
```

```
1 };
```

```
const cartAfterAdd = [...initialCart,  
newItem];
```

```
// Б. УДАЛЕНИЕ (Delete item) → .filter() по ID:
```

```
const removeId =
```

```
2;
```

```
// Удаляем мышь
```

```
const cartAfterDelete =
```

```
cartAfterAdd.filter(item ⇒ item.id !==  
removeId);
```

```
// В. ИЗМЕНЕНИЕ (Update item) → .map() с проверкой ID:
```

```
const targetId =
```

```
1;
```

```
// Увеличиваем количество клавиатур до 2
```

```
const cartAfterUpdate =
```

```
cartAfterDelete.map(item ⇒ item.id ===  
targetId ? { ...item, quantity: item.quantity +
```

```
1 } : item );
```

```
// Г. ПОДСЧЕТ ИТОГА (Compute total) → .reduce():
```

```
const totalPrice =
```

```
cartAfterUpdate.reduce((acc, item) =>
```

```
acc + item.price * item.quantity, 0);
```

```
console.log("Итоговая корзина:", cartAfterUpdate); console.log(`Общая сумма: ${totalPrice} руб.`); // 15 000 р
```

Живая песочница иммутабельных операций

Client Runtime Simulator

Нажимайте кнопки для симуляции действий пользователя в стейте:

+ Добавить товар

🗑 Удалить товар ID: 2

↑ Увеличить кол-во ID: 1

↺ Сбросить

```
// Состояние стейта: [Клавиатура (x1), Мышь (x2), Коврик (x1)] // Итоговая сумма: 10 000 руб. |
```

```
Элементов: 3
```